

5) `split()` and `rsplit()`

`split()` is used to remove specific element from string.

using `maxsplit` we can differentiate `split()` & `rsplit()`.

6) `dir()` function.

is used to find all attributes present inside a datatype.

7) `a = tuple()`

`type(a)`

`<class 'tuple'>`

* List Data Type :-

- It is a collection of homogeneous & heterogeneous characters.

Homogeneous $\rightarrow$ same type of data

Heterogeneous $\Rightarrow$ Different types of data

- list data type is order.

- It will accept duplicate value.

- It is mutable. we can modify original data.

Syntax =
`Var: name = [e1, e2, e3 ...]`

1) How to create empty string list?

`vn = []`

`a = []`

`o/p = []`

& using object

`vn = list()`

`o/p = []`

Slicing in list:-

Syntax = `vn [si:ei+1:sv]`

indexing & slicing

eg- `x = ["hello", "python", "walmart"]`

`x[0]`

`"hello"`

`# 10`

`x[0][3:5:1]`

`# mar`

`x[2][2:6:1]`

2) `x = [["akash", "Rahul"]]`

`x[0]`

`["akash", "Rahul"]`

`x[0][0]`

`"akash"`

`x[0][0][1]`

`'Rahul'`

list Methods :-

dir(list)

1) ADDING ELEMENT TO THE LIST :

- 1 Append()
- 2 Extend()
- 3 Insert()

1) append()

- It is used to add element to list at last position.

- We can use both datatype i.e. Collection & individual.

- It is not remove boundary condition.

- return none (modifies original list in back)

Syntax :- `vn.append(element)`

eg:

```
l = [4, 'active', 'hello']
```

```
l.append(45.5)
```

```
# return none
```

```
l
```

```
[4, 'active', 'hello', 45.5]
```

2) extend()

- It is used to add the element to the list at end.

- It removes boundary condition.

- We can use only collection data type.

Syntax :- `vn.extend(elements)`
`vn.extend(iterable)`

eg:- a = [1, 2, 3]

a.extend(45) [↑] it's not
Type error multi value data

a.extend("python")

a

[1, 2, 3, 'P', 'Y', 't', 'h', 'o', 'n']

a.extend([[56]])

a

[1, 2, 3, 'P', 'Y', 't', 'h', 'o', 'n', [56]]

3) insert()

- It is used to add the element inside the list at any position.
- We can use both type of data collection & individual data type.

Syntax:-

vn.insert(position, value)

eg: n = [12, 100, 300]

If position not

n.insert(1, "python") = present it add at last.

[12, 100, 300] [12, "python", 100, 300]

reverse()

Syntax:- vn.reverse()

- It will reverse the sequence of list.
- It affects the original data.
- It will not reverse the nested list.

Removing elements from the list

1. pop()
2. remove()
3. clear()

1. pop()

It is used to remove element from list at any position.

If you don't specify any index (position) it will remove last index value (-1).

If position is not present it throw INDEX ERROR.

It return removed value.

Syntax :- $vn.pop() \rightarrow$ internally it work.

$vn.pop(position)$

eg :-

$l = ['hello', 'good', 'morning', 'people']$

$l.pop()$ — By default it will remove -1 Position.

$['hello', 'good', 'morning']$

$l.pop()$

$['good']$

2) remove()

- It is used to remove element from list
- If you don't specify any index it will throw TYPE ERROR
- If specified value is not present it throw VALUE ERROR

Syntax:-

mandatory field
↓
vn.remove(element)

eg:-

```
l = ["hello", "good", "morning", "people"]
```

```
l.remove()
```

Type Error:

```
l.remove("morning")
```

return none

```
["hello", "good", "people"]
```

3) clear()

- It will clear all element inside list.

(Boundary condition will

not remove)

- return empty list

Syntax:-

vn.clear()

eg: l.clear()

```
[]
```

4) Del keyword

It will remove entire list from memory location

del

Syntax = del var-name.

7) Count ()

It is used to count of specified element which is present inside a list.

If specified element is not present inside list, it will return 0.

Syntax :- `list.count (element)` or `vn.count (element)`

8) index ()

It will return index number / position of specified element, which is present inside list.

It will return index number of first occurrence of specified element.

Value error if posil element is not present

Syntax :- `vn.index (element)`

eg:

`l = [75, 45, 95, 75, 15, 25, 45, 75]`

`l.index(15)`

4

`l.index(100)`

ValueError

Copy

It is used to copy value present in one variable to another variable.

Copy operation can be performed in 3 ways.

1. normal copy

2. shallow copy

3. deepcopy

1. normal copy()

- address of copy list and original list is always same.

- address of original nested list and copied nested list will be same

Syntax =

new vn = original vn.

a = [1, 2, 3, 4]

b = a

a

[1, 2, 3, 4]

b

[1, 2, 3, 4]

id(a)

2122252969088

id(b)

2122252969088

note:- 1) Address is always same then change in one list affects the others list

2) If address is different then change in one does not affects other list.

2) Shallow copy

- We will be using inbuilt method `copy()`

Syntax: `new_var = original_var.copy()`

- address of original list and copied list will be different.

address of original nested list and copied nested list will be same.

`x = [11, 12, 13]`

`y = x.copy()`

`x`
`[11, 12, 13]`

`y`
`[11, 12, 13]`

`id(x)`
`id(y)` $\rightarrow$ both different

#nested Shallow copy()

`b = [11, 12, 13, ['a', 'b', 'c']]`

`v = b.copy()`

`b`
`[11, 12, 13, ['a', 'b', 'c']]`

`v`
`[11, 12, 13, ['a', 'b', 'c']]`

`id(b)`
`id(v)` $\rightarrow$ both different

`id(b[3])`
`id(v[3])` $\rightarrow$ both same Nested list

10) deepcopy()

- deepcopy() is method present inside copy module.
- We have to import it.

Syntax :-

```
from copy import deepcopy
new-Var = deepcopy(original variable)
```

- address of original list and copied list will be different.
- address of original nested list and copied nested list will be different.

11) Sort()

- sort will be sort the list by default it sort list in ascending order.
- In order to sort list, list should be homogeneous.
- It modifies original list itself.

Syntax :- `vn.sort()`

✓ `vn.sort(key=None, reverse=False)` as to do
 ✓ `vn.sort(key=len, reverse=False)` True - d to A.

Note: If we want to sort list in descending order we have to change REVERSE=False to REVERSE=True.

ASCII value

American Standard Code for Information Interchange. A-Z (65-90) a-z (97-122)
0-9 → 48-57

ASCII value C inbuilt function

↓
`ord()`

↓
`chr()`

- ordinary

character

- It will always

accepts one

- syntax:-

character with quotes

`chr("Number")`

syntax:-

`ord('character')`

- No. → character

O/P :- Number

- No. without quotes.

e → N

eg: `chr(65)`

O/P :- 'A'

O/p number is
ascii no.

eg: `ord('A')`

O/P :- 65

** 1)

What is meaning of sequenced data types

→ which type of data we can perform indexing & slicing

All collection data type is iterable.

But all iterable not a sequence data type
(in set & dict we can't do indexing & slicing)

All sequence data type are iterable